Query & Database Optimization Exercises - GameVerse Database

Duration: 3-4 hours
Database: GameVerse (Game Store Database)
Topics: EXPLAIN / EXPLAIN ANALYZE, Query rewriting, Indexing, Partitioning, Clustering

---

Before You Start

These exercises are designed to show measurable performance differences. The existing GameVerse dataset is small, so you will first generate extra data. Do not skip the setup section.

All exercises follow the same pattern:

1. Run a query **before** the optimization and record the plan and timing.
2. Apply the optimization (index, rewrite, partition, etc.).
3. Run the same query **after** and compare.

Always record both plans in your answer.

Setup: Scaling the GameVerse Dataset

Run the script below once before starting. It adds data up to the defined number of users and the probabilities set. The script works in any PostgreSQL client (psql, pgAdmin, DBeaver, etc.).

```
-- ============================================================
-- generate_data.sql
-- Synthetic data generator for the GameVerse optimization exercises
-- ============================================================
--
-- Strategy: insert N users, then for each table cross-join users
-- with games / achievements and keep each pair with probability p.
-- This guarantees all FK references are valid (IDs come directly
-- from the real tables) and requires no temp tables or complex logic.
--
-- Approximate row counts with default probabilities
-- (based on 50,000 users, 37 games, 28 achievements):
--
--   user_library     : 50,000 * 37 * 0.108  ≈    200,000 rows
--   reviews          : 50,000 * 37 * 0.054  ≈    100,000 rows
--   user_achievements: 50,000 * 28 * 0.357  ≈    500,000 rows
--
-- Adjust the probabilities below to scale the dataset up or down.
-- ============================================================

-- Author: Víctor Barceló
```

```
SET app.num_users       = '100000';
SET app.p_library       = '0.108';
SET app.p_reviews       = '0.054';
SET app.p_achievements  = '0.357';

-- Performance settings for bulk loading.
-- synchronous_commit = off: skip WAL fsync after each statement (biggest
speedup).
--   Data is still crash-safe once COMMIT completes.
-- work_mem: gives the CROSS JOIN hash join more memory to avoid disk
spills.
SET synchronous_commit = off;
SET work_mem = '128MB';

-- Truncate all affected tables so this script can be re-run cleanly.
-- RESTART IDENTITY resets all SERIAL sequences back to 1.
-- CASCADE handles FK-dependent tables in the correct order automatically.
-- WARNING: this removes ALL existing rows, including the seed data from
insert_data.sql.
-- Re-run insert_data.sql afterwards if you need the original seed rows
back.
TRUNCATE users, user_library, reviews, user_achievements RESTART IDENTITY
CASCADE;

-- Wrap all inserts in a single transaction: one fsync instead of four.
BEGIN;

-- ============================================================
-- 1. Users
-- ============================================================
INSERT INTO users (
    username, email, registration_date, country, birth_date,
    role, account_status, is_premium, last_login, total_spent
)
SELECT
    'user_' || i,
    'user_' || i || '@gameverse.com',
    DATE '2018-01-01' + (random() * 2557)::int,
    CASE (i % 6)
        WHEN 0 THEN 'ES' WHEN 1 THEN 'US' WHEN 2 THEN 'FR'
        WHEN 3 THEN 'DE' WHEN 4 THEN 'JP' ELSE 'BR'
    END,
    DATE '1980-01-01' + (random() * 15000)::int,
    CASE (i % 20) WHEN 0 THEN 'moderator' WHEN 1 THEN 'analyst' ELSE
'user' END,
    CASE (i % 30) WHEN 0 THEN 'suspended' ELSE 'active' END,
    (i % 4 = 0),
    NOW() - ((random() * 365)::int || ' days')::interval,
    (random() * 2000)::numeric(10,2)
FROM generate_series(1, current_setting('app.num_users')::int) i;

-- ============================================================
-- 2. User library  (random subset of games per user)
-- ============================================================
```

```
-- Each (user, game) pair is kept with probability p_library.
-- The WHERE random() < p filter is evaluated once per combination,
-- so every user ends up owning a different random selection of games.
INSERT INTO user_library (user_id, game_id, purchase_date, purchase_price,
hours_played)
SELECT
    u.user_id,
    g.game_id,
    DATE '2018-01-01' + (random() * 2557)::int,
    (random() * 70 + 0.99)::numeric(6,2),
    (random() * 5000)::numeric(8,2)
FROM users  u
CROSS JOIN games g
WHERE random() < current_setting('app.p_library')::numeric;

-- ============================================================
-- 3. Reviews  (random subset of games per user)
-- ============================================================
INSERT INTO reviews (user_id, game_id, rating, review_text, review_date,
helpful_count)
SELECT
    u.user_id,
    g.game_id,
    (random() * 9 + 1)::int,
    'Synthetic review by user ' || u.user_id || ' for game ' || g.game_id,
    DATE '2018-01-01' + (random() * 2557)::int,
    (random() * 500)::int
FROM users  u
CROSS JOIN games g
WHERE random() < current_setting('app.p_reviews')::numeric;

-- ============================================================
-- 4. User achievements  (random subset of achievements per user)
-- ============================================================
INSERT INTO user_achievements (user_id, achievement_id, unlocked_date)
SELECT
    u.user_id,
    a.achievement_id,
    NOW() - ((random() * 1000)::int || ' days')::interval
FROM users        u
CROSS JOIN achievements a
WHERE random() < current_setting('app.p_achievements')::numeric;

COMMIT;

-- Restore defaults.
SET synchronous_commit = on;
SET work_mem = '4MB';

-- ============================================================
-- 5. Refresh planner statistics
-- ============================================================
ANALYZE users, user_library, reviews, user_achievements;
```

```
-- ============================================================
-- 6. Verify row counts
-- ============================================================
SELECT 'users'            AS tbl, COUNT(*) FROM users
UNION ALL
SELECT 'user_library',           COUNT(*) FROM user_library
UNION ALL
SELECT 'reviews',                COUNT(*) FROM reviews
UNION ALL
SELECT 'user_achievements',      COUNT(*) FROM user_achievements;
```

Part 1: Reading Execution Plans with EXPLAIN

Exercise 1.1 - Your First EXPLAIN

Run the query below using both `EXPLAIN` and `EXPLAIN ANALYZE`:

```
SELECT * FROM users WHERE country = 'ES';
```

Tasks:

a) Which scan type is used - `Seq Scan` or `Index Scan`?
b) What is the estimated row count (`rows=`) vs. the actual row count?
c) What is the actual execution time reported by `EXPLAIN ANALYZE`?
d) What does the `cost=start..total` figure represent?

▶ Hint

```
EXPLAIN SELECT * FROM users WHERE country = 'ES';
EXPLAIN ANALYZE SELECT * FROM users WHERE country = 'ES';
```

Focus on the top-level node in the output. The `cost` values are in arbitrary planner units, not milliseconds. The actual time is in milliseconds and only appears with `ANALYZE`.

Exercise 1.2 - Estimations vs. Reality

```
SELECT * FROM reviews WHERE rating = 10;
```

a) Run `EXPLAIN ANALYZE`. How many rows does the planner **estimate** vs. how many actually come back?
b) If the estimate is very different from reality, what PostgreSQL command should you run to help the planner?
c) Run that command and then `EXPLAIN ANALYZE` again. Did the estimates improve?

▶ Hint

The command that refreshes planner statistics is `ANALYZE table_name;`. After running it, the planner has fresh column histograms and the estimates should be much closer to the actual counts.

Exercise 1.3 - Interpreting Join Nodes

```
SELECT u.username, g.title, ul.hours_played
FROM user_library ul
JOIN users  u ON ul.user_id  = u.user_id
JOIN games  g ON ul.game_id  = g.game_id
WHERE ul.hours_played > 1000;
```

a) Run `EXPLAIN ANALYZE`. List every node type that appears in the plan (e.g., `Hash Join`, `Seq Scan`, `Index Scan`).
b) Which table is on the "build" side of any Hash Join? Which is on the "probe" side?
c) What does the `loops=` counter mean in the EXPLAIN output?

Part 2: Query Rewriting for Better Performance

Exercise 2.1 - Correlated Subquery vs. JOIN

The following query retrieves each user's total hours played using a correlated subquery:

```
SELECT
    u.username,
    (SELECT SUM(ul.hours_played)
     FROM user_library ul
     WHERE ul.user_id = u.user_id) AS total_hours
FROM users u
WHERE u.account_status = 'active'
LIMIT 1000;
```

Tasks:

a) Run `EXPLAIN ANALYZE` and record the execution time.
b) Rewrite the query using a `JOIN` and `GROUP BY` instead of the correlated subquery.
c) Run `EXPLAIN ANALYZE` on your rewritten version. Is it faster? Why?

▶ Hint - rewritten structure

```
SELECT u.username, SUM(ul.hours_played) AS total_hours
FROM users u
LEFT JOIN user_library ul ON u.user_id = ul.user_id
WHERE u.account_status = 'active'
```

```
GROUP BY u.username
LIMIT 1000;
```

Exercise 2.2 - Avoiding Function Calls on Filtered Columns

```
SELECT * FROM users WHERE LOWER(country) = 'es';
```

a) Run `EXPLAIN ANALYZE`. Can PostgreSQL use any index on `country` here?
b) Why does wrapping a column in a function prevent index use?
c) Rewrite the query so it can use a plain index on `country`.
d) As an advanced alternative, create a **functional index** and confirm it is used:

```
CREATE INDEX idx_users_country_lower ON users (LOWER(country));
```

Run `EXPLAIN` again after creating the functional index. What changed?

Exercise 2.3 - SELECT * vs. Required Columns

```
-- Version A
EXPLAIN ANALYZE SELECT * FROM user_achievements WHERE user_id = 100;

-- Version B
EXPLAIN ANALYZE SELECT achievement_id, unlocked_date FROM
user_achievements WHERE user_id = 100;
```

a) Run both versions. Is there a difference in the plan or execution time?
b) After you create the index in Part 3 (Exercise 3.1), come back and re-run both. Does Version B benefit more than Version A? Why?

Exercise 2.4 - EXISTS vs. COUNT for Existence Checks

You want to know whether a user has any reviews. Compare:

```
-- Version A: using COUNT
SELECT user_id
FROM users
WHERE (SELECT COUNT(*) FROM reviews r WHERE r.user_id = users.user_id) > 0
LIMIT 500;

-- Version B: using EXISTS
SELECT user_id
```

```
FROM users
WHERE EXISTS (SELECT 1 FROM reviews r WHERE r.user_id = users.user_id)
LIMIT 500;
```

a) Run `EXPLAIN ANALYZE` on both.
b) Which is faster and why does `EXISTS` short-circuit?

Exercise 2.5 - CTE Inlining (PostgreSQL 12+)

```
-- Using a plain CTE
WITH premium_users AS (
    SELECT user_id, username, total_spent
    FROM users
    WHERE is_premium = TRUE
)
SELECT pu.username, COUNT(r.review_id) AS review_count
FROM premium_users pu
LEFT JOIN reviews r ON pu.user_id = r.user_id
GROUP BY pu.username
ORDER BY review_count DESC
LIMIT 20;
```

a) Run `EXPLAIN ANALYZE`. Is the CTE inlined (does the planner push the `is_premium` filter down) or materialized?
b) Force materialization:

```
WITH premium_users AS MATERIALIZED ( ... )
```

c) Compare both plans. Which is faster and why?

Part 3: Indexing

Exercise 3.1 - Basic B-tree Index

```
-- Baseline: no index on user_id in user_achievements
EXPLAIN ANALYZE
SELECT achievement_id, unlocked_date
FROM user_achievements
WHERE user_id = 100;
```

a) Record the scan type and execution time.

```
CREATE INDEX idx_ua_user_id ON user_achievements (user_id);
```

b) Run the same `EXPLAIN ANALYZE` again. What changed?
c) What is the approximate speedup factor?

Exercise 3.2 - Composite Index and Column Order

```
-- Query filters on both game_id and rating
EXPLAIN ANALYZE
SELECT review_id, rating, review_date
FROM reviews
WHERE game_id = 5 AND rating >= 8;
```

a) Run the query before any new index. Record the plan.
b) Create two candidate indexes and test each:

```
-- Option 1: game_id first
CREATE INDEX idx_reviews_game_rating ON reviews (game_id, rating);

-- Option 2: rating first
CREATE INDEX idx_reviews_rating_game ON reviews (rating, game_id);
```

c) For each option, run `EXPLAIN ANALYZE`. Which index is chosen by the planner?
d) Now run a query that only filters on `rating >= 8` (no `game_id`). Which of the two indexes does the planner prefer?
e) What rule about column order in composite indexes does this demonstrate?

▶ Hint

The **left-prefix rule**: a composite index on `(A, B)` can be used for queries that filter on `A` alone, or on both `A` and `B`. It cannot efficiently serve queries that filter on `B` alone.

Exercise 3.3 - Partial Index

Many background jobs poll for suspended users. Only ~3% of users have `account_status = 'suspended'`.

```
-- Baseline
EXPLAIN ANALYZE
SELECT user_id, username, email
FROM users
WHERE account_status = 'suspended';
```

a) Record the plan.

```
CREATE INDEX idx_users_suspended ON users (user_id)
WHERE account_status = 'suspended';
```

b) Run `EXPLAIN ANALYZE` again. Is the partial index used?
c) Check the size of the partial index vs. a full index on the same column:

```
CREATE INDEX idx_users_status_full ON users (account_status);

SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexname::regclass)) AS size
FROM pg_indexes
WHERE tablename = 'users'
  AND indexname IN ('idx_users_suspended', 'idx_users_status_full');
```

d) Why is the partial index smaller? When would you prefer a partial index over a full one?

Exercise 3.4 - Covering Index (INCLUDE)

```
EXPLAIN ANALYZE
SELECT user_id, total_spent, is_premium
FROM users
WHERE country = 'US';
```

a) Create a plain index: `CREATE INDEX idx_users_country ON users (country);`
Run `EXPLAIN ANALYZE`. Does the plan show `Index Scan` or `Index Only Scan`?

b) Drop that index and create a covering index:

```
CREATE INDEX idx_users_country_cover ON users (country)
INCLUDE (total_spent, is_premium);
```

Run `EXPLAIN ANALYZE` again. Did it switch to `Index Only Scan`? What is the benefit?

▶ Hint

An `Index Only Scan` means PostgreSQL retrieved all needed columns directly from the index without visiting the table heap. This saves I/O because the heap fetch is skipped. The `INCLUDE` clause stores extra columns in the index leaf pages without making them part of the sort key.

Exercise 3.5 - Finding and Removing Unused Indexes

a) Create two indexes that will not be used by any query in this session:

```
CREATE INDEX idx_unused_helpful ON reviews (helpful_count);
CREATE INDEX idx_unused_website  ON publishers (website);
```

b) Run a few queries that do **not** touch these columns.

c) Query the index usage statistics view:

```
SELECT schemaname, tablename, indexname, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY tablename, indexname;
```

d) Drop the unused indexes. Why is it important to remove indexes that are never used?

Part 4: Partitioning

For these exercises you will create a new partitioned version of `reviews` to isolate the work from the main table.

Exercise 4.1 - Range Partitioning by Date

```
-- Step 1: Create the partitioned parent table
CREATE TABLE reviews_partitioned (
    review_id    BIGSERIAL,
    user_id      INT NOT NULL,
    game_id      INT NOT NULL,
    rating       INT CHECK (rating BETWEEN 1 AND 10),
    review_text  TEXT,
    review_date  DATE NOT NULL DEFAULT CURRENT_DATE,
    helpful_count INT DEFAULT 0
) PARTITION BY RANGE (review_date);

-- Step 2: Create yearly partitions
CREATE TABLE reviews_2018 PARTITION OF reviews_partitioned
    FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');
CREATE TABLE reviews_2019 PARTITION OF reviews_partitioned
    FOR VALUES FROM ('2019-01-01') TO ('2020-01-01');
CREATE TABLE reviews_2020 PARTITION OF reviews_partitioned
    FOR VALUES FROM ('2020-01-01') TO ('2021-01-01');
CREATE TABLE reviews_2021 PARTITION OF reviews_partitioned
    FOR VALUES FROM ('2021-01-01') TO ('2022-01-01');
CREATE TABLE reviews_2022 PARTITION OF reviews_partitioned
    FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');
```

```
CREATE TABLE reviews_2023 PARTITION OF reviews_partitioned
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
CREATE TABLE reviews_2024 PARTITION OF reviews_partitioned
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE reviews_default PARTITION OF reviews_partitioned DEFAULT;

-- Step 3: Copy data from the original table
INSERT INTO reviews_partitioned (review_id, user_id, game_id, rating,
review_text, review_date, helpful_count)
SELECT review_id, user_id, game_id, rating, review_text, review_date,
helpful_count
FROM reviews;

ANALYZE reviews_partitioned;
```

Tasks:

a) Run EXPLAIN on a date-filtered query against the **unpartitioned** table and record the plan:

```
EXPLAIN SELECT COUNT(*) FROM reviews WHERE review_date >= '2023-01-01';
```

b) Run the same query against the **partitioned** table:

```
EXPLAIN SELECT COUNT(*) FROM reviews_partitioned WHERE review_date >=
'2023-01-01';
```

How many partitions appear in the plan? This is **partition pruning** - the planner skips irrelevant partitions entirely.

c) Add an index on `game_id` to the partitioned table:

```
CREATE INDEX idx_rp_game_id ON reviews_partitioned (game_id);
```

Confirm the index was automatically created on each child partition:

```
SELECT indexname, tablename FROM pg_indexes
WHERE tablename LIKE 'reviews_%'
ORDER BY tablename, indexname;
```

d) Run `EXPLAIN ANALYZE` for a query that combines both the partition key and another column:

```
EXPLAIN ANALYZE
SELECT rating, helpful_count
```

```
FROM reviews_partitioned
WHERE review_date BETWEEN '2023-01-01' AND '2023-12-31'
  AND game_id = 5;
```

Which partitions are scanned? Is the index on `game_id` used within the selected partition?

Exercise 4.2 - List Partitioning by Country

```
CREATE TABLE users_by_region (
    user_id          BIGSERIAL,
    username         VARCHAR(50) NOT NULL,
    email            VARCHAR(100) NOT NULL,
    country          CHAR(2)     NOT NULL,
    registration_date DATE,
    is_premium       BOOLEAN DEFAULT FALSE,
    total_spent      NUMERIC(10,2) DEFAULT 0
) PARTITION BY LIST (country);

CREATE TABLE users_region_europe   PARTITION OF users_by_region
    FOR VALUES IN ('ES', 'FR', 'DE', 'IT', 'PT', 'NL');
CREATE TABLE users_region_americas PARTITION OF users_by_region
    FOR VALUES IN ('US', 'CA', 'MX', 'BR', 'AR');
CREATE TABLE users_region_apac     PARTITION OF users_by_region
    FOR VALUES IN ('JP', 'CN', 'AU', 'KR', 'IN');
CREATE TABLE users_region_other    PARTITION OF users_by_region DEFAULT;

INSERT INTO users_by_region (user_id, username, email, country,
registration_date, is_premium, total_spent)
SELECT user_id, username, email, country, registration_date, is_premium,
total_spent
FROM users;

ANALYZE users_by_region;
```

Tasks:

a) Run `EXPLAIN` for a query filtering on a single country:

```
EXPLAIN SELECT COUNT(*) FROM users_by_region WHERE country = 'ES';
```

How many partitions are scanned?

b) Run `EXPLAIN` for a query with no country filter:

```
EXPLAIN SELECT COUNT(*) FROM users_by_region;
```

What happens to partition pruning when there is no filter on the partition key?

c) Try inserting a user from a country not in any list partition (e.g., `'ZZ'`). Which partition receives it?

Exercise 4.3 - Partition Management: Detach and Drop

Starting from `reviews_partitioned`:

a) Detach the 2018 partition without deleting it:

```
ALTER TABLE reviews_partitioned DETACH PARTITION reviews_2018;
```

Confirm the table still exists and contains rows.

b) Compare the time to delete old data two ways:

```
-- Method A: DELETE on the unpartitioned table
-- (do not actually run this if your dataset is large - just EXPLAIN it)
EXPLAIN DELETE FROM reviews WHERE review_date < '2019-01-01';

-- Method B: drop the detached partition (instantaneous)
DROP TABLE reviews_2018;
```

c) Why is dropping a partition orders of magnitude faster than `DELETE`?

Exercise 4.4 - Hash Partitioning

```
CREATE TABLE user_achievements_hashed (
    user_id        INT NOT NULL,
    achievement_id INT NOT NULL,
    unlocked_date  TIMESTAMP DEFAULT NOW()
) PARTITION BY HASH (user_id);

CREATE TABLE ua_hash_0 PARTITION OF user_achievements_hashed FOR VALUES
WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE ua_hash_1 PARTITION OF user_achievements_hashed FOR VALUES
WITH (MODULUS 4, REMAINDER 1);
CREATE TABLE ua_hash_2 PARTITION OF user_achievements_hashed FOR VALUES
WITH (MODULUS 4, REMAINDER 2);
CREATE TABLE ua_hash_3 PARTITION OF user_achievements_hashed FOR VALUES
WITH (MODULUS 4, REMAINDER 3);

INSERT INTO user_achievements_hashed
SELECT user_id, achievement_id, unlocked_date FROM user_achievements;

ANALYZE user_achievements_hashed;
```

Tasks:

a) Check the row distribution across partitions:

```
SELECT tableoid::regclass AS partition, COUNT(*) AS row_count
FROM user_achievements_hashed
GROUP BY tableoid
ORDER BY tableoid::regclass::text;
```

Is the distribution roughly even?

b) Run `EXPLAIN` for a point lookup on a specific `user_id`:

```
EXPLAIN SELECT * FROM user_achievements_hashed WHERE user_id = 42;
```

How many partitions are scanned? (Should be exactly 1.)

c) Run `EXPLAIN` for a query with no `user_id` filter:

```
EXPLAIN SELECT COUNT(*) FROM user_achievements_hashed;
```

What do you observe about the number of partitions scanned?

d) What is the main advantage of hash partitioning compared to range and list partitioning?

Part 5: Clustering

Exercise 5.1 - Basic CLUSTER Operation

The `user_library` table is inserted in no particular order, so rows for the same `user_id` are scattered across many heap pages. A common query fetches all library entries for one user:

```
-- Baseline: before clustering
EXPLAIN (ANALYZE, BUFFERS)
SELECT game_id, purchase_date, hours_played
FROM user_library
WHERE user_id = 500;
```

a) Record the number of `Buffers: shared hit` and `shared read` pages.

```
CREATE INDEX IF NOT EXISTS idx_ul_user_id ON user_library (user_id);
CLUSTER user_library USING idx_ul_user_id;
```

```
ANALYZE user_library;
```

b) Run the same `EXPLAIN (ANALYZE, BUFFERS)` query again. How did the buffer counts change?
c) What does it mean for a table to be "clustered"? Does the cluster order persist after new rows are inserted?

Exercise 5.2 - Observing Cluster Decay

a) After clustering `user_library` on `user_id`, insert 200,000 new rows with random `user_id` values:

> If you are running this exercise in a fresh session (the Setup script was run in a previous session), recreate the lookup tables first:
>
> ```
> SET app.num_decay = '200000';
> CREATE TEMP TABLE _uid AS
>     SELECT user_id, row_number() OVER (ORDER BY user_id)::int AS rn
> FROM users;
> CREATE TEMP TABLE _gid AS
>     SELECT game_id, row_number() OVER (ORDER BY game_id)::int AS rn
> FROM games;
> CREATE INDEX ON _uid (rn);
> CREATE INDEX ON _gid (rn);
> -- Also drop any leftover random-pick table from a previous run:
> DROP TABLE IF EXISTS _rnd_decay;
> ```

```
-- Rows to insert for decay simulation (set at top of Setup, or override
here)
SET app.num_decay = '200000';

CREATE TEMP TABLE _rnd_decay AS
SELECT
    (1 + floor(random() * (SELECT COUNT(*) FROM _uid))::int) AS uid_rn,
    (1 + floor(random() * (SELECT COUNT(*) FROM _gid))::int) AS gid_rn
FROM generate_series(1, current_setting('app.num_decay')::int);

INSERT INTO user_library (user_id, game_id, purchase_date, purchase_price,
hours_played)
SELECT
    u.user_id,
    g.game_id,
    DATE '2024-01-01' + (random() * 365)::int,
    (random() * 70 + 0.99)::numeric(6,2),
    0
FROM _rnd_decay r
JOIN _uid u ON u.rn = r.uid_rn
JOIN _gid g ON g.rn = r.gid_rn
ON CONFLICT DO NOTHING;
```

```
DROP TABLE _rnd_decay;
```

b) Re-run the `EXPLAIN (ANALYZE, BUFFERS)` query from Exercise 5.1. Did the buffer counts increase?
c) Re-cluster and run a third time. What do you observe?

```
CLUSTER user_library;   -- re-uses the previously recorded index
ANALYZE user_library;
```

d) In a production environment, how would you keep a heavily written table clustered efficiently?

▶ Hint

`CLUSTER` acquires an exclusive lock on the table (blocking reads and writes during re-ordering). For production use, the extension **pg_repack** can perform the same physical reordering online without a prolonged lock. A common strategy is to schedule `CLUSTER` or `pg_repack` during maintenance windows.

Exercise 5.3 - Choosing the Right Column to Cluster On

The `reviews` table is queried in two very different ways:

- **Pattern A**: `WHERE game_id = ?` (find all reviews for one game)
- **Pattern B**: `WHERE review_date BETWEEN ? AND ?` (find reviews in a date range)

a) Create both candidate indexes:

```
CREATE INDEX idx_reviews_game   ON reviews (game_id);
CREATE INDEX idx_reviews_date   ON reviews (review_date);
```

b) Run `EXPLAIN (ANALYZE, BUFFERS)` for both Pattern A and Pattern B **before clustering**.

c) Cluster on `game_id` and re-run both patterns. Record buffer counts.

d) Cluster on `review_date` and re-run both patterns. Record buffer counts.

e) Which clustering choice gives the best overall performance? How would you decide in a real project?

Part 6: Bringing It All Together

Exercise 6.1 - Full Optimization Pipeline

You receive a complaint that the following report query is slow. It calculates the top 10 most-reviewed games in 2023 among premium users, including the average rating.

```

```

```sql
-- Step 0: Baseline - run as-is and record the plan
EXPLAIN ANALYZE
SELECT
    g.title,
    COUNT(r.review_id)  AS review_count,
    AVG(r.rating)::numeric(4,2) AS avg_rating
FROM reviews r
JOIN users  u ON r.user_id  = u.user_id
JOIN games  g ON r.game_id  = g.game_id
WHERE u.is_premium  = TRUE
  AND r.review_date BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY g.title
ORDER BY review_count DESC
LIMIT 10;
```

Apply the following optimizations **one at a time**, running `EXPLAIN ANALYZE` after each step:

Step 1 - Index the join columns and filter columns:

```sql
CREATE INDEX idx_reviews_user  ON reviews (user_id);
CREATE INDEX idx_reviews_date2 ON reviews (review_date);
CREATE INDEX idx_users_premium ON users (user_id) WHERE is_premium = TRUE;
```

Step 2 - Add a covering index to avoid heap fetches on reviews:

```sql
CREATE INDEX idx_reviews_cover ON reviews (review_date, user_id, game_id,
rating)
WHERE review_date BETWEEN '2023-01-01' AND '2023-12-31';
```

Step 3 - Use the partitioned reviews table (from Exercise 4.1) so the planner prunes to the 2023 partition automatically.

Step 4 - Create a materialized view for the report so it only runs once and is cached:

```sql
CREATE MATERIALIZED VIEW mv_top_games_2023 AS
SELECT
    g.title,
    COUNT(r.review_id)       AS review_count,
    AVG(r.rating)::numeric(4,2) AS avg_rating
FROM reviews_partitioned r
JOIN users  u ON r.user_id  = u.user_id
JOIN games  g ON r.game_id  = g.game_id
WHERE u.is_premium  = TRUE
  AND r.review_date BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY g.title
ORDER BY review_count DESC
```

```
LIMIT 10;

CREATE UNIQUE INDEX ON mv_top_games_2023 (title);

-- Query the materialized view
SELECT * FROM mv_top_games_2023;

-- Refresh when underlying data changes (does not block reads)
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_top_games_2023;
```

Deliverable: Fill in the table below with your measured times:

Step	Execution Time	Key Change
Baseline		No indexes or partitioning
After Step 1		Join and filter indexes
After Step 2		Covering index
After Step 3		Partition pruning
After Step 4		Materialized view

Reflection Questions

1. When should you choose **not** to add an index to a large, frequently-written table?
2. You have a query `WHERE UPPER(username) = 'ALICE'`. A regular index on `username` exists but is not being used. What two solutions could fix this?
3. Explain in your own words why partition pruning reduces I/O. Under what condition does pruning **not** apply?
4. After running `CLUSTER`, a colleague says "we're done, the table will stay fast forever." What is wrong with this statement?
5. A composite index exists on `(game_id, rating)`. Which of the following queries can use it efficiently? Justify each answer.
    - `WHERE game_id = 5`
    - `WHERE rating = 9`
    - `WHERE game_id = 5 AND rating = 9`
    - `WHERE game_id > 10 AND rating < 5`
6. Compare `CLUSTER` with creating an index. Both can improve query performance. What does each one do differently at the storage level?
7. You need to delete all reviews older than 2020. You have two options: `DELETE FROM reviews WHERE review_date < '2020-01-01'` vs. dropping a partition. Which do you choose and why?

8. When would a **materialized view** be better than a regular view for a reporting query? What is the main trade-off?